

1 · Python

(a)

[1]

[1, 2]

(b) Argumento padrão mutável. O `[]` é criado uma única vez, na definição da função, e a mesma lista é reutilizada nas chamadas seguintes. Liga-se ao ambiente local (§9.4): o padrão fica "preso" à função, como uma variável estática.

2 · Java

(a) `0 5`

(b) Em Java tudo é passado por valor (§9.5.4).

- `v` é um objeto: a função recebe uma cópia da referência, mas ela aponta para o mesmo array, então `v[0] = 0` é visto pelo chamador.
- `n` é primitivo: a função recebe uma cópia do valor, então `n = 0` só altera a cópia local.

3 · Python

(a) `[2, 2, 2]`

(b) Fechamentos (§9.12). O fechamento captura a variável (a célula de memória), e não uma cópia do valor. Existe um único `i`, que termina o laço valendo 2, então as três lambdas devolvem 2. É o mesmo caso do `var` em JavaScript, que dava `[3, 3, 3]`. Para capturar o valor de cada volta, use `lambda i=i: i`.

4 · C

(a) 3

(b) Variável local estática (`static`, §9.4). Ela fica na memória estática, e não no registro de ativação na pilha. Por isso inicializa uma vez só e lembra o valor entre chamadas. Sem o `static`, a saída seria 1.

5 · Rust

(a) Não compila. Erro E0382: `borrow of moved value: v`.

(b) Passagem por posse (`move`). Ao chamar `dobra(v)`, a posse do vetor passa para a função, e `v` deixa de ser válido no chamador. O `println!` tenta usar `v` depois do `move`. Correções:

- passar emprestado: `fn dobra(v: &Vec<i32>)` e chamar `dobra(&v)`
- ou passar uma cópia: `dobra(v.clone())`

Se corrigido, a saída seria `[1, 2, 3] [2, 4, 6]`.

6 · Python

(a) `UnboundLocalError: local variable 'total' referenced before assignment`

(b) Ambiente de referenciamento local e escopo (§9.4). Como há uma atribuição a `total` dentro da função, o Python trata `total` como local em todo o corpo. Assim, o `total + x` lê uma local ainda sem valor, e o `total = 0` global não é usado. Correções:

- declarar `global total` dentro da função (o `nonlocal` da síntese serve para variáveis de uma função externa, não para globais)
- ou, melhor, sem efeito colateral: `def adiciona(x, total): return total + x`